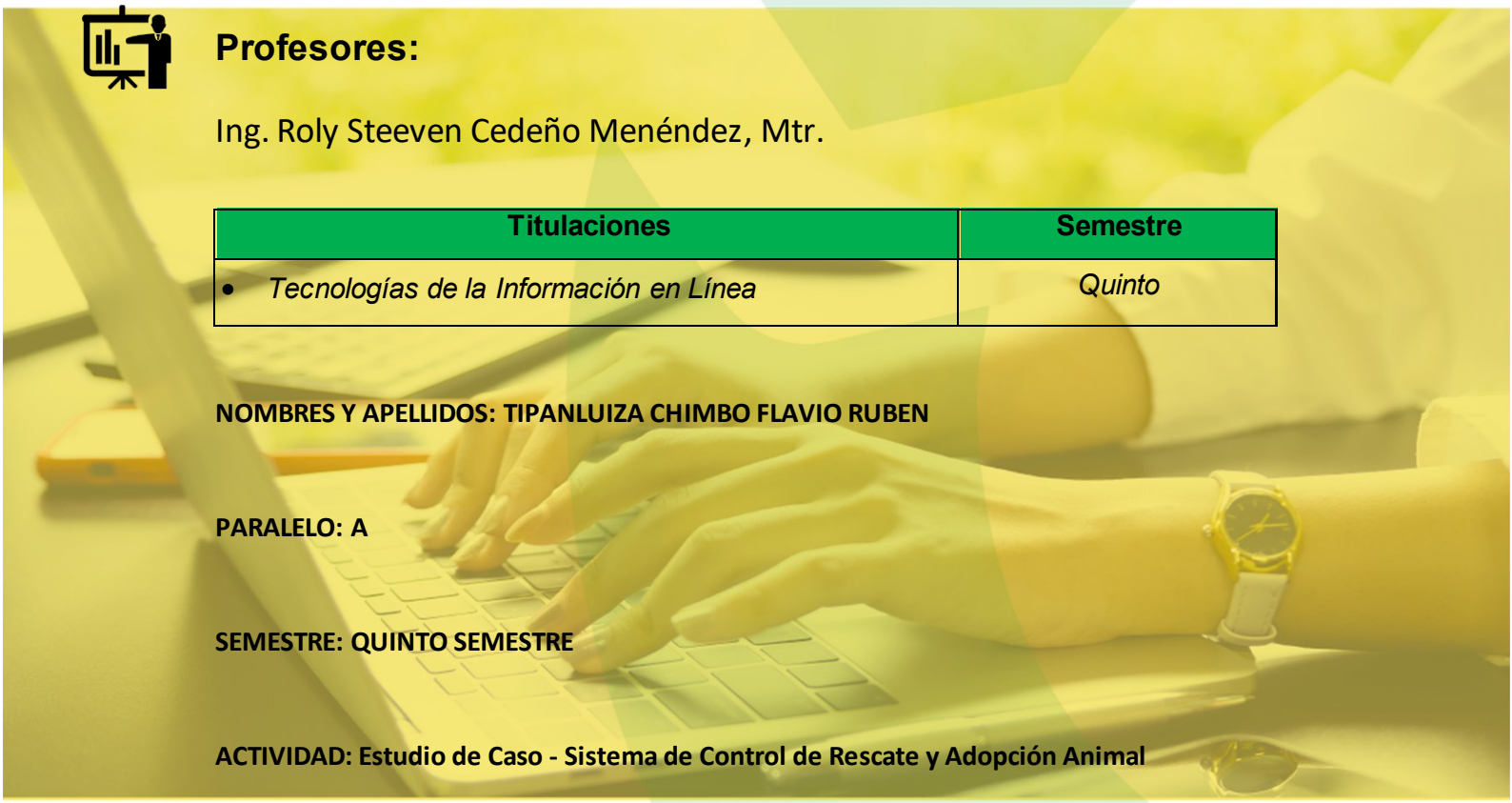
UTM online

Universidad Técnica de Manabí desde 1952

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: TIPANLUIZA CHIMBO FLAVIO RUBEN

PARALELO: A

SEMESTRE: QUINTO SEMESTRE

ACTIVIDAD: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2026 - ENERO 2027

Actividades a desarrollar en la Unidad _1___

Actividad # _1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

1. Introducción

El desarrollo de sistemas informáticos requiere una adecuada planificación de su estructura, de manera que los diferentes componentes puedan organizarse, mantenerse y modificarse conforme evolucionan las necesidades del proyecto. En este contexto, el diseño de software permite definir de forma anticipada cómo interactuarán los elementos de un sistema y cómo se distribuirán sus responsabilidades.

La presente actividad propone el diseño de un Sistema de Control de Rescate y Adopción Animal, orientado a la gestión de casos dentro de un refugio. El sistema permitirá registrar el ingreso de animales rescatados, clasificarlos según su especie, asignar voluntarios o veterinarios responsables y controlar los diferentes estados del proceso, tales como Rescatado, En Tratamiento y Adopción. Estas características corresponden directamente al escenario establecido en la guía de la actividad.

Para organizar adecuadamente la solución se emplearán diagramas UML, principios SOLID, patrones de diseño de software y un esquema de arquitectura que permita separar responsabilidades. Además, el desarrollo del proyecto será controlado mediante un repositorio público en GitHub utilizando el modelo Gitflow, con el propósito de mantener un registro ordenado de los cambios realizados durante el proceso.

2. Objetivos

2.1 Objetivo general

Diseñar la arquitectura lógica de un Sistema de Control de Rescate y Adopción Animal, aplicando patrones de diseño de software, principios SOLID, diagramas UML y un flujo de trabajo basado en Gitflow, con la finalidad de estructurar una solución escalable, reutilizable y mantenible.

Este objetivo mantiene el propósito planteado por la guía, que establece el diseño de una arquitectura lógica aplicando patrones y flujos profesionales de trabajo.

2.2 Objetivos específicos

- Identificar los actores que intervienen en el proceso de rescate y adopción animal.
- Definir los requerimientos funcionales y no funcionales del sistema.
- Diseñar diagramas UML que representen las interacciones y estructura del sistema.
- Aplicar principios SOLID en la definición de las clases y responsabilidades.
- Aplicar los patrones de diseño Singleton, Observer, Factory Method y MVC.
- Diseñar una arquitectura general que integre los componentes principales del sistema.
- Utilizar un repositorio público en GitHub mediante el flujo Gitflow.
- Documentar el proceso mediante capturas de ramas, commits y avances del proyecto.

3. Análisis del problema

3.1 Descripción del problema

Los refugios y organizaciones dedicadas al rescate de animales requieren administrar información relacionada con los diferentes casos que atienden. Cada rescate puede involucrar animales de distintas especies, personas que reportan la situación, voluntarios encargados de apoyar el proceso y veterinarios responsables de la atención médica.

Cuando esta información no se encuentra organizada dentro de un sistema, puede existir dificultad para conocer el estado actual de cada caso, identificar quién se encuentra a cargo del animal y mantener un seguimiento adecuado desde el momento del rescate hasta su recuperación o adopción.

Por esta razón, se plantea el diseño de un sistema que permita centralizar la información de los rescates realizados por el refugio. El sistema deberá permitir registrar animales rescatados, clasificarlos como perros, gatos o fauna silvestre, asignar responsables y gestionar diferentes estados del proceso. Estos elementos son expresamente solicitados en la descripción de la actividad.

Además, el sistema deberá ser diseñado de manera que pueda ampliarse y mantenerse con facilidad. Para lograrlo, se utilizarán patrones de diseño, principios SOLID y herramientas de control de versiones.

3.2 Actores principales

La guía identifica como actores principales al Ciudadano/Reportante, Voluntario/Veterinario y Administrador del Refugio.

Para representar mejor sus responsabilidades dentro del sistema se consideran los siguientes actores:

Actor	Descripción
Ciudadano / Reportante	Persona que reporta un animal que requiere rescate y proporciona la información inicial del caso.
Voluntario	Persona que participa en las actividades de rescate, traslado y seguimiento del animal.
Veterinario	Profesional encargado de evaluar el estado del animal y registrar información relacionada con su tratamiento.
Administrador del Refugio	Responsable de gestionar los casos, asignar responsables y controlar el estado de cada proceso.

3.3 Requerimientos funcionales

La guía menciona expresamente como requerimientos funcionales: crear casos de rescate, categorizar especies y asignar voluntarios.

Código	Requerimiento funcional
RF01	El sistema permitirá registrar un nuevo caso de rescate.
RF02	El sistema permitirá registrar los datos del animal rescatado.
RF03	El sistema permitirá clasificar al animal según su especie.
RF04	El sistema permitirá diferenciar entre perro, gato y fauna silvestre.
RF05	El sistema permitirá asignar un voluntario a un caso.
RF06	El sistema permitirá asignar un veterinario cuando el animal requiera atención médica.
RF07	El sistema permitirá modificar el estado del caso.
RF08	El sistema permitirá manejar estados como Rescatado, En Tratamiento y Adopción.

RF09	El sistema permitirá consultar los casos registrados.
RF10	El sistema permitirá consultar la información de cada animal.
RF11	El sistema permitirá notificar a los voluntarios sobre nuevos rescates.
RF12	El administrador podrá gestionar la asignación de responsables.

3.4 Requerimientos no funcionales

La guía establece como requerimientos no funcionales principales la escalabilidad, mantenibilidad y trazabilidad de versiones.

Código	Requerimiento no funcional
RNF01	El sistema deberá ser escalable para permitir la incorporación de nuevas especies y funcionalidades.
RNF02	El sistema deberá ser mantenible mediante una estructura organizada y modular.
RNF03	Los cambios realizados deberán tener trazabilidad mediante Git y GitHub.
RNF04	La arquitectura deberá separar claramente las responsabilidades de cada componente.
RNF05	Las clases y componentes deberán diseñarse para facilitar su reutilización.
RNF06	Los nombres de clases y métodos deberán ser claros y significativos.

4. Diseño del sistema

4.1 Diagrama de casos de uso

El diagrama de casos de uso permitirá representar las principales interacciones entre los actores y el Sistema de Control de Rescate y Adopción Animal.

Actores que deben aparecer

- Ciudadano / Reportante
- Voluntario
- Veterinario
- Administrador del Refugio

Casos de uso que vamos a incluir

1. Reportar animal

2. Registrar caso de rescate
3. Registrar datos del animal
4. Categorizar especie
5. Consultar casos
6. Asignar voluntario
7. Asignar veterinario
8. Actualizar estado del caso
9. Registrar tratamiento
10. Recibir notificación de nuevo rescate
11. Gestionar adopción
12. Gestionar casos

Relación entre actores y casos de uso

Actor	Casos de uso
Ciudadano / Reportante	Reportar animal
Voluntario	Consultar casos, recibir notificación, actualizar seguimiento
Veterinario	Consultar caso, registrar tratamiento, actualizar estado
Administrador	Registrar caso, categorizar especie, asignar voluntario, asignar veterinario, actualizar estado, gestionar adopción y gestionar casos

El diagrama representa las principales interacciones entre los usuarios y el sistema. El ciudadano inicia el proceso mediante el reporte de un animal, mientras que el administrador registra y gestiona el caso. Los voluntarios y veterinarios intervienen posteriormente de acuerdo con las responsabilidades asignadas.

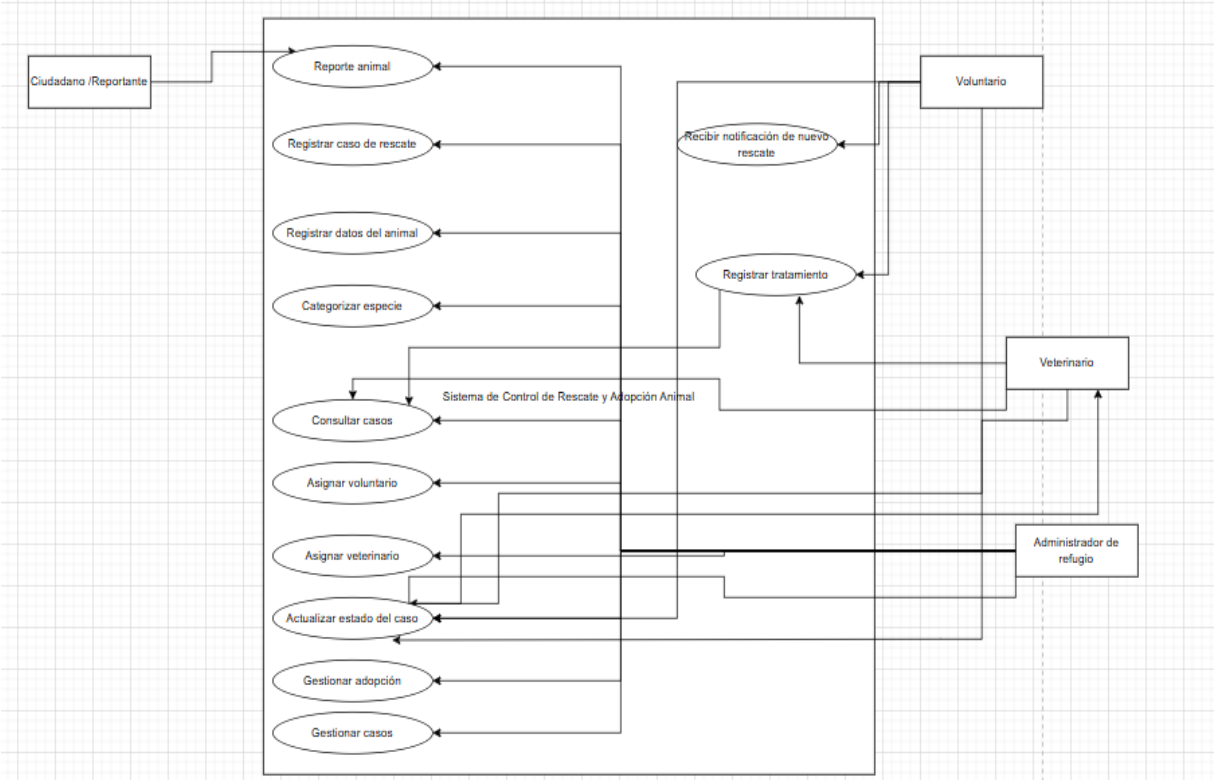


Figura 1. Diagrama de casos de uso del Sistema de Control de Rescate y Adopción Animal.

4.2 Descripción de casos de uso

La descripción de los casos de uso permite detallar las principales acciones que pueden realizar los diferentes actores dentro del Sistema de Control de Rescate y Adopción Animal, cada caso de uso representa una función específica del sistema y establece la forma en que el Ciudadano/Reportante, Voluntario, Veterinario y Administrador del Refugio interactúan con la plataforma.

Para facilitar la comprensión del funcionamiento propuesto, se describen a continuación los casos de uso representados en el diagrama:

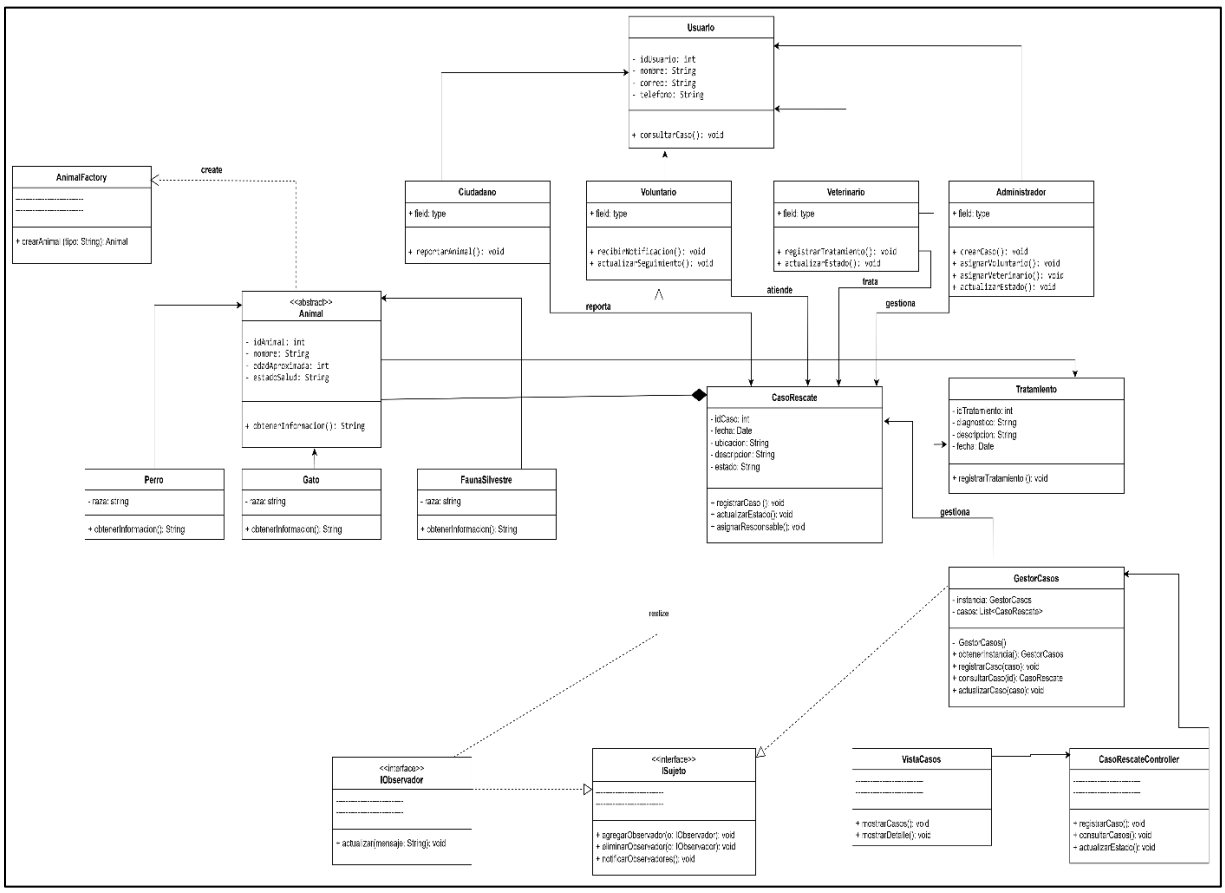
Código	Caso de uso	Actor principal	Descripción
CU01	Reportar animal	Ciudadano / Reportante	Permite comunicar la existencia de un animal que requiere rescate.
CU02	Registrar caso de rescate	Administrador	Permite crear el expediente inicial del caso.
CU03	Registrar datos del animal	Administrador	Registra información básica del animal rescatado.

CU04	Categorizar especie	Administrador	Permite clasificar al animal como perro, gato o fauna silvestre.
CU05	Asignar voluntario	Administrador	Asigna un responsable voluntario al caso.
CU06	Asignar veterinario	Administrador	Asigna un veterinario cuando el caso necesita atención médica.
CU07	Consultar casos	Voluntario / Veterinario / Administrador	Permite revisar los casos registrados.
CU08	Actualizar estado	Voluntario / Veterinario / Administrador	Modifica el estado del proceso.
CU09	Registrar tratamiento	Veterinario	Registra información relacionada con la atención del animal.
CU10	Recibir notificación	Voluntario	Informa sobre la existencia de un nuevo rescate.
CU11	Gestionar adopción	Administrador	Permite gestionar el proceso cuando el animal se encuentra disponible para adopción.
CU12	Gestionar casos	Administrador	Permite administrar el conjunto de casos registrados.

4.3 Diagrama de clases UML

El diagrama de clases UML permite representar la estructura lógica del Sistema de Control de Rescate y Adopción Animal, identificando las principales clases, sus atributos, métodos y relaciones. Para el presente diseño se consideran los actores y procesos definidos previamente, procurando distribuir las responsabilidades de manera clara y aplicando principios SOLID que faciliten la mantenibilidad, reutilización y escalabilidad del sistema.

Para la construcción del modelo se consideran clases relacionadas con los usuarios, los animales, los casos de rescate, los tratamientos y la gestión de notificaciones.



4.4 Aplicación de principios SOLID

Los principios SOLID permiten organizar el diseño del software de manera que cada componente tenga responsabilidades claras y pueda modificarse o ampliarse sin afectar innecesariamente a otras partes del sistema; En el Sistema de Control de Rescate y Adopción Animal, estos principios se aplican mediante la separación de funciones entre las diferentes clases y el uso de abstracciones.

Principio de Responsabilidad Única – SRP

Cada clase tiene una función específica dentro del sistema, por citar, **CasoRescate** se encarga de administrar la información correspondiente a un rescate, Tratamiento gestiona la información médica del animal y **AnimalFactory** se ocupa de la creación de los diferentes tipos de animales.

Justificación:

Esta separación evita que una misma clase concentre demasiadas funciones, facilitando el mantenimiento y la comprensión del sistema.

Principio Abierto/Cerrado – OCP

La clase abstracta **Animal** permite incorporar nuevos tipos de animales mediante nuevas clases derivadas sin modificar su estructura original.

Por ejemplo:

Animal→Perro

Animal→Gato

Animal → FaunaSilvestre

Justificación:

Si posteriormente el refugio necesita registrar aves, reptiles u otra especie, se puede crear una nueva clase sin alterar las clases existentes.

Principio de Sustitución de Liskov – LSP

Las clases **Perro**, **Gato** y **FaunaSilvestre** pueden utilizarse como objetos de tipo Animal, manteniendo el comportamiento esperado definido por la clase principal.

Justificación:

Esto permite que el sistema trabaje de forma general con objetos de tipo Animal, independientemente de la especie específica.

Principio de Segregación de Interfaces – ISP

Las interfaces **IObservador** e **ISujeto** contienen únicamente los métodos necesarios para gestionar el sistema de notificaciones.

IObservador contiene:

+ actualizar(mensaje: String): void

Mientras que ISujeto contiene:

+ agregarObservador(o: IObservador): void

+ eliminarObservador(o: IObservador): void

+ notificarObservadores(): void

Justificación:

De esta manera, las clases implementan solamente las funciones que realmente necesitan y se evita incorporar métodos innecesarios.

Principio de Inversión de Dependencias – DIP

El sistema de notificaciones se apoya en interfaces como **IObservador** e **ISujeto**, evitando que las clases principales dependan directamente de implementaciones específicas.

Justificación:

Esto reduce el acoplamiento entre los componentes y facilita que en el futuro se puedan agregar diferentes mecanismos de notificación sin modificar la estructura principal.

Justificación general de SOLID

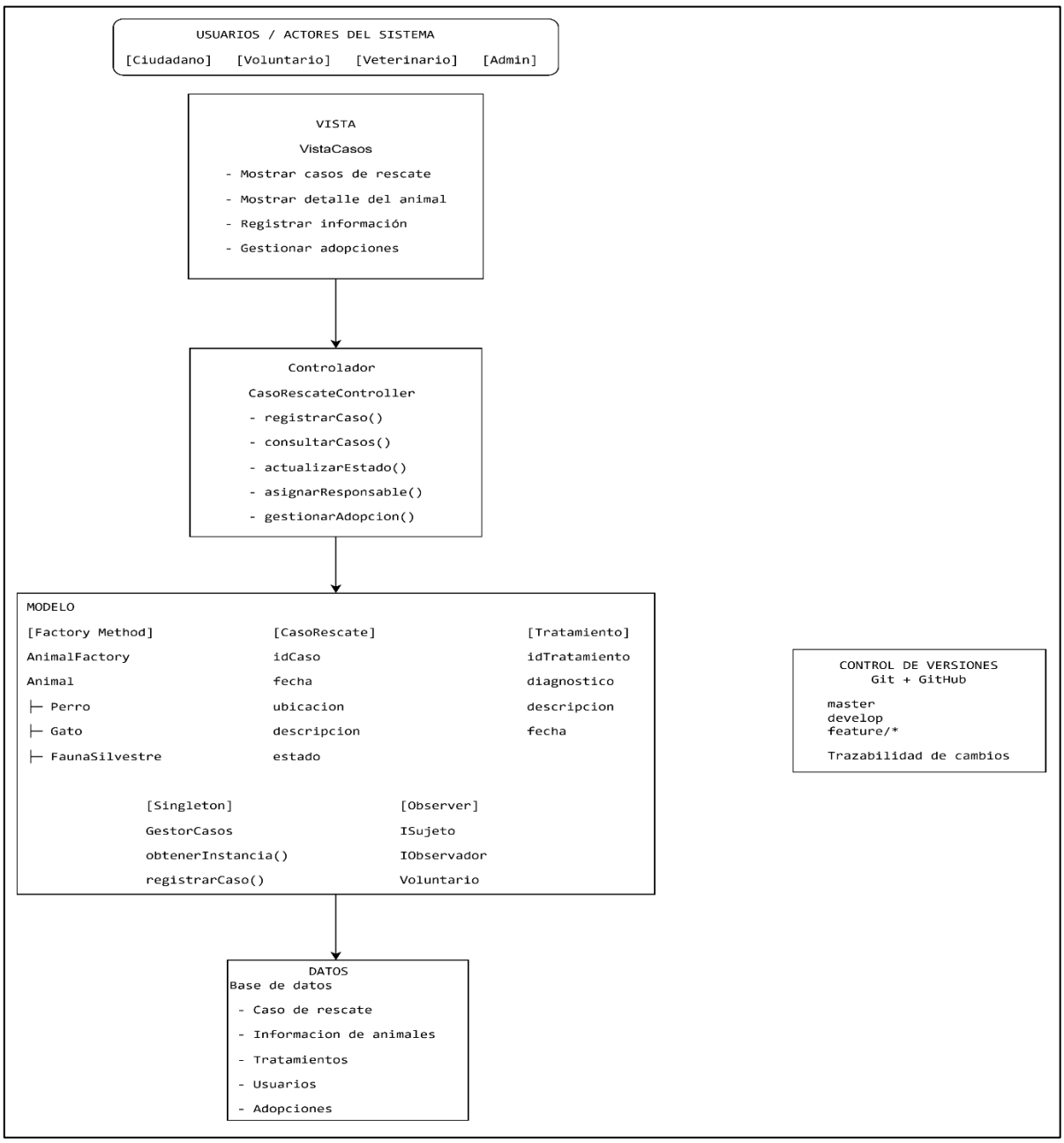
La aplicación de los principios SOLID permite que la arquitectura propuesta sea más organizada, flexible y fácil de mantener. La separación de responsabilidades, el uso de herencia, interfaces y abstracciones favorecen la incorporación de nuevas funcionalidades sin realizar modificaciones extensas en el sistema. De esta forma, el diseño mantiene coherencia con los requerimientos de escalabilidad y mantenibilidad establecidos para la actividad.

4.5 Arquitectura general del sistema

La arquitectura general del Sistema de Control de Rescate y Adopción Animal se plantea bajo el patrón **MVC (Model-View-Controller)**, con la finalidad de separar la interfaz de usuario, la lógica de control y los datos del sistema. Esta organización facilita el mantenimiento del software, permite distribuir mejor las responsabilidades y contribuye a una estructura más clara y escalable.

La capa de **Vista** será responsable de presentar la información al usuario y permitir la interacción con el sistema. La capa de **Controlador** recibirá las acciones realizadas desde la interfaz y coordinará las operaciones correspondientes. Finalmente, la capa de **Modelo** contendrá las clases principales encargadas de gestionar los datos y la lógica del sistema, como **CasoRescate**, **Animal**, **Tratamiento**, **GestorCasos** y las demás clases asociadas.

Dentro de esta arquitectura se integran también los patrones de diseño seleccionados. **AnimalFactory** permitirá crear diferentes tipos de animales mediante Factory Method; **GestorCasos** centralizará la administración de los casos mediante Singleton; y el mecanismo de notificaciones se apoyará en Observer para informar a los voluntarios cuando se registre un nuevo rescate.



La arquitectura propuesta mediante el patrón MVC permite separar las responsabilidades del sistema en diferentes componentes. La Vista se encarga de la interacción con el usuario, el Controlador procesa las solicitudes y coordina las operaciones, mientras que el Modelo administra la información y la lógica relacionada con los casos de rescate, animales, tratamientos y responsables.

La integración de Singleton, Factory Method y Observer complementa la arquitectura al resolver necesidades específicas del sistema. Singleton permite centralizar la gestión de los casos, Factory Method facilita la creación de distintos tipos de animales y

Observer permite notificar automáticamente a los voluntarios cuando se registra un nuevo rescate.

Finalmente, la incorporación de Gitflow permite mantener un control ordenado de los cambios y versiones del proyecto, proporcionando trazabilidad durante el desarrollo y facilitando la integración de nuevas funcionalidades mediante ramas independientes.

5. Aplicación de patrones de diseño

Para el Sistema de Control de Rescate y Adopción Animal se emplean cuatro patrones de diseño que permiten organizar la solución y distribuir correctamente las responsabilidades entre sus componentes. Cada patrón responde a una necesidad específica del sistema y contribuye a mejorar la reutilización, mantenibilidad y escalabilidad de la arquitectura.

5.1 Patrón Singleton

Propósito

El patrón Singleton permite garantizar que exista una única instancia de una clase durante la ejecución del sistema.

Aplicación en el sistema

En este proyecto se aplicará sobre la clase:

GestorCasos

- instancia: GestorCasos
- casos: List<CasoRescate>

- GestorCasos()
+ obtenerInstancia(): GestorCasos
+ registrarCaso(): void
+ consultarCaso(): CasoRescate
+ actualizarCaso(): void

Problema que resuelve

Sin este patrón podrían existir varias instancias del gestor administrando información diferente. Singleton permite que todos los componentes del sistema trabajen con un mismo punto de gestión de casos.

Justificación

Se utiliza Singleton porque el sistema necesita mantener un control centralizado de los casos de rescate. De esta forma, las operaciones de registro, consulta y actualización pueden realizarse a través de una única instancia del gestor, evitando duplicidad en la administración de la información.

5.2 Patrón Observer

Propósito

El patrón Observer permite que varios objetos sean informados automáticamente cuando ocurre un cambio o evento determinado.

Aplicación en el sistema

Este patrón se utilizará en el proceso de notificaciones.

Cuando se registre un nuevo caso de rescate:

Nuevo rescate

↓

GestorCasos

↓

Notificar observadores

↓

Voluntarios

Las principales interfaces serán:

<<interface>>

ISujeto

+ agregarObservador()

+ eliminarObservador()

+ notificarObservadores()

y:

<<interface>>

IObservador

+ actualizar(mensaje: String): void

La clase Voluntario implementará IObservador.

Problema que resuelve

Evita que el sistema tenga que contactar manualmente a cada voluntario cada vez que se registre un rescate.

Justificación

Observer permite implementar un mecanismo automático de notificaciones. Cuando se registre un nuevo caso, el sistema podrá informar a los voluntarios registrados sin generar una dependencia directa entre el caso de rescate y cada voluntario.

5.3 Patrón Factory Method

Propósito

Factory Method permite crear distintos tipos de objetos sin que el componente que los solicita tenga que conocer los detalles específicos de su creación.

Aplicación en el sistema

Se utilizará la clase:

```
AnimalFactory
-----------------------------
+ crearAnimal(tipo: String): Animal
```

Dependiendo del tipo recibido, podrá generar:

```
AnimalFactory
    │
    ├── Perro
    ├── Gato
    └── FaunaSilvestre
```

Por ejemplo, conceptualmente:

Si tipo = "Perro"
→ crear Perro

Si tipo = "Gato"
→ crear Gato

Si tipo = "Fauna Silvestre"
→ crear FaunaSilvestre

Problema que resuelve

Evita colocar repetidamente condiciones y procesos de creación de objetos en diferentes partes del sistema.

Justificación

Factory Method facilita la creación de diferentes tipos de animales desde un único punto. Además, permite incorporar nuevas especies en el futuro sin alterar significativamente la lógica de los componentes existentes.

5.4 Justificación general de los patrones seleccionados

Justificación general

La aplicación de los patrones Singleton, Observer y Factory Method permite resolver diferentes necesidades dentro del Sistema de Control de Rescate y Adopción Animal. Singleton centraliza la gestión de los casos, Observer permite generar notificaciones automáticas para los voluntarios, Factory Method facilita la creación de diferentes tipos de animales y MVC organiza la solución mediante la separación de la interfaz, la lógica de control y el modelo de datos.

La utilización conjunta de estos patrones permite construir una arquitectura más organizada y flexible, disminuyendo el acoplamiento entre componentes y facilitando futuras modificaciones o ampliaciones del sistema. Cada patrón cumple una función específica y se integra con los requerimientos previamente definidos para el proyecto.

6. Diagrama de secuencia

6.1 Propósito

El diagrama de secuencia permite representar el orden en que interactúan los diferentes participantes del sistema durante un proceso específico.

Para esta actividad utilizaremos como escenario principal:

Registro de un nuevo rescate y notificación a voluntarios.

Este escenario permite mostrar claramente la intervención de:

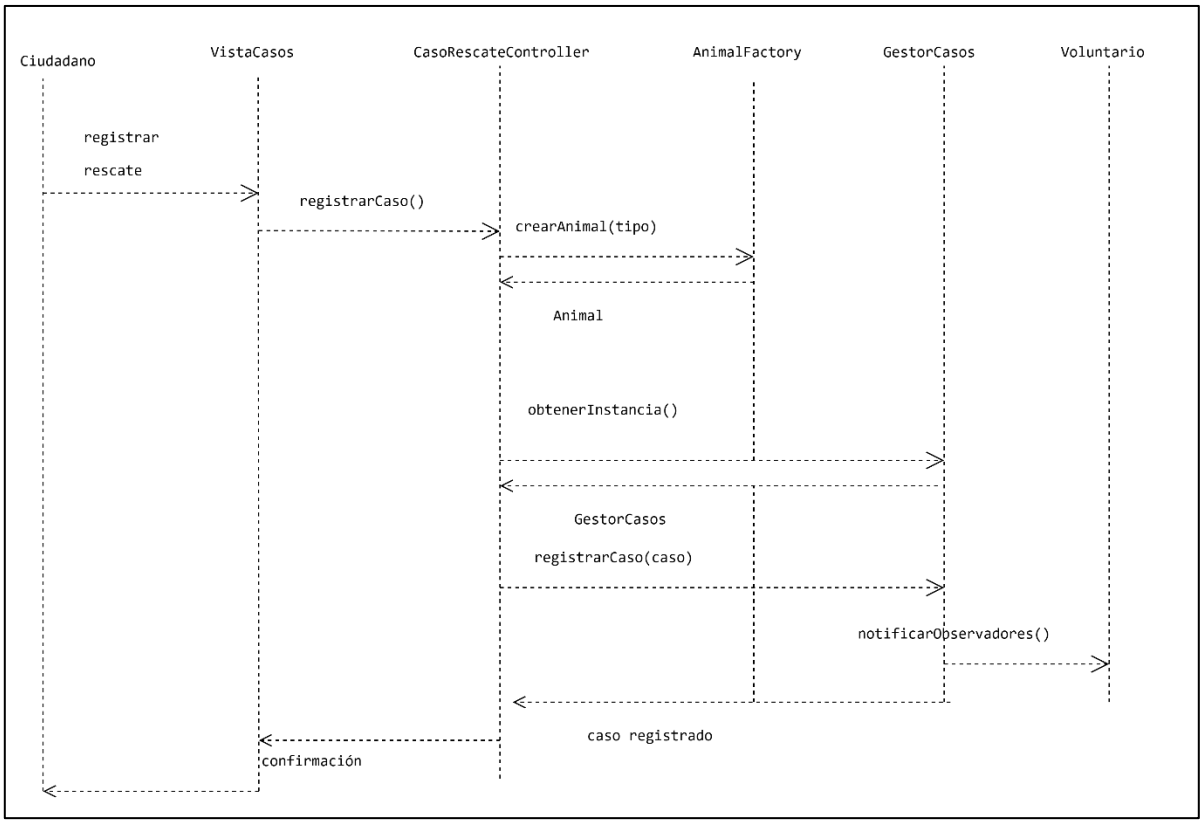
- Ciudadano / Reportante
- VistaCasos
- CasoRescateController
- AnimalFactory
- GestorCasos
- Voluntario

y al mismo tiempo evidencia la aplicación de Factory Method, Singleton y Observer.

6.2 Secuencia que vamos a representar

El flujo será el siguiente:

1. El Ciudadano/Reportante registra la información de un animal que necesita rescate.
2. La VistaCasos recibe los datos.
3. La vista envía la solicitud al CasoRescateController.
4. El controlador solicita a AnimalFactory crear el animal según su tipo.
5. AnimalFactory crea el objeto correspondiente: Perro, Gato o FaunaSilvestre.
6. El controlador obtiene la instancia de GestorCasos.
7. GestorCasos registra el nuevo caso de rescate.
8. GestorCasos ejecuta la notificación a los observadores.
9. El Voluntario recibe la notificación del nuevo rescate.
10. El sistema confirma al ciudadano que el caso fue registrado.



El diagrama de secuencia representa el proceso de registro de un nuevo caso de rescate animal. El flujo comienza cuando el Ciudadano/Reportante proporciona la información del animal mediante la interfaz del sistema. La Vista envía los datos al Controlador, encargado de coordinar las operaciones necesarias.

Posteriormente, el Controlador utiliza AnimalFactory para crear el objeto correspondiente según la especie reportada. Una vez creado el animal, se obtiene la instancia de GestorCasos, responsable de registrar el nuevo caso dentro del sistema. Después del registro, GestorCasos ejecuta el mecanismo de notificación para informar a los voluntarios sobre la existencia de un nuevo rescate. Finalmente, el sistema devuelve una confirmación de que el caso ha sido registrado correctamente.

Evidencias

(Código de la ejecución de los ejercicios, enlace drive donde se encuentren los mismos y Capturas de pantalla claras y relevantes, donde evidencie el proceso realizado).

https://drive.google.com/file/d/1BwE5JWB9RV-PRpQXQ7_eJSMtZ8sC70tM/view?usp=drive_link

https://drive.google.com/file/d/1_3tcaaeoBt3h5azc6K-NK7Qzo3C1QWzD/view?usp=drive_link

https://drive.google.com/file/d/1hJrGu7SP3T8N_XVLeg20rceWotRn6bBm/view?usp=drive_link

https://drive.google.com/file/d/1FXCZigAM5ZgK37TX8PRitfXzvLaLIT1Q/view?usp=drive_link

UCRT64:/i/Mi unidad/5TO SEMESTRE 2026/TAREAS/DSI/TAREAS DSI/TAREA NO. 1

```
flavi@FLAVIO UCRT64 /i/Mi unidad/5TO SEMESTRE 2026/TAREAS/DSI/TAREAS DSI/TAREA NO. 1 (main)
$ git init
Initialized empty Git repository in I:/Mi unidad/5TO SEMESTRE 2026/TAREAS/DSI/TAREAS DSI/TAREA NO. 1/.git/

flavi@FLAVIO UCRT64 /i/Mi unidad/5TO SEMESTRE 2026/TAREAS/DSI/TAREAS DSI/TAREA NO. 1 (master)
$ git add .
git commit -m "commit inicial: subida de la primera version de la presentacionn"
git commit -m "commit update: subida del archivo adicional tutorial"
On branch master

Initial commit

nothing to commit (create/copy files and use "git add" to track)
On branch master

Initial commit

nothing to commit (create/copy files and use "git add" to track)

flavi@FLAVIO UCRT64 /i/Mi unidad/5TO SEMESTRE 2026/TAREAS/DSI/TAREAS DSI/TAREA NO. 1 (master)
$ ^[[200~git remote add origin https://github.com/ftipanluiza3846-max/tarea-utm2026.git
bash: $'\E[200~git': command not found
git branch -M main
git push -u origin maingit branch -M main
git push -u origin main
flavi@FLAVIO UCRT64 /i/Mi unidad/5TO SEMESTRE 2026/TAREAS/DSI/TAREAS DSI/TAREA NO. 1 (master)
$ git branch -M main
git push -u origin maingit push -u origin maingit push -u origin main
flavi@FLAVIO UCRT64 /i/Mi unidad/5TO SEMESTRE 2026/TAREAS/DSI/TAREAS DSI/TAREA NO. 1 (main)
$ git push -u origin main~
```

UCRT64:/c/Users/flavi

```
flavi@FLAVIO UCRT64 ~
$ git config --global user.name "FLAVIO"

flavi@FLAVIO UCRT64 ~
$ git config --global user.email "ftipanluiza3846@utm.edu.ec"

flavi@FLAVIO UCRT64 ~
$ git config --global --list
user.name=FLAVIO
user.email=ftipanluiza3846@utm.edu.ec

flavi@FLAVIO UCRT64 ~
$ git remote add origin https://github.com/ftipanluiza3846-max/tarea-utm2026.git
git branch -M main
git push -u origin main
fatal: not a git repository (or any of the parent directories): .git
fatal: not a git repository (or any of the parent directories): .git
fatal: not a git repository (or any of the parent directories): .git

flavi@FLAVIO UCRT64 ~
$
```

Bibliografía

(Referencias bibliográficas y/o enlaces en Internet (normas APA) que se utilizaron para realizar la práctica.)